HANDS-ON LAB 04 GUIDE & COPY-PASTE PROMPTS • OMNICHANNEL RETAIL EDITION (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 04 (EN)

Lab 04: Multi-Agent Capstone — Google ADK, A2A Protocol, Omnichannel Routing & Gemini Enterprise Deployment

Duration: 45 Minutes | **Persona**: Technical & Commercial Track | **Prerequisites**: Labs 01, 02 & 03 Completed

1. Retail Scenario & Business Context

In Labs 01–03, your team stabilized promotional margin floors, repaired True ATP inventory calculations, encoded retail compliance standards (**`GEMINI.md`**, **`.agents/rules/retail-standards.md`**, **`.agents/skills/retail-otif-margin-auditor/SKILL.md`**), and connected Antigravity to **`retail_pulse.db`** via FastMCP and deterministic **`PreToolUse`** hooks.

However, two major operational challenges remain:

1. **35 VIP orders (`DIAMOND` and `GOLD` loyalty tiers)** representing **over $250,000 in Customer Lifetime Value (LTV)** are still stuck in **`AT_RISK_STOCKOUT`** status because their original flagship store node lacked sufficient True ATP inventory.
2. Enterprise retail operations require **specialist AI agents** — a **Technical WMS/POS Inventory Agent** and a **Commercial Merchandising Margin & OTIF Agent** — that coordinate via **Google ADK (Agent Development Kit)**, expose an open **A2A (Agent2Agent Protocol)** endpoint (**`/.well-known/agent.json`** and **`POST /api/v1/a2a`**), can be inspected locally in the **ADK Web UI (`adk web`)**, and deploy to **Google Cloud Run + Gemini Enterprise** via **Terraform**.

2. Learning Objectives

In this capstone lab, you will:

1. Use **`/plan`**, **`/boost`**, and **`/teamwork`** to implement **`evaluate_candidates`** and **`optimize_at_risk_orders`** in **`src/services/fulfillment_router.py`**, rerouting all 35 **`AT_RISK_STOCKOUT`** VIP orders to **`REROUTED_OPTIMAL`** and preserving **`$14,774.00`** in post-shipping net margin.
2. Design and implement a **Multi-Agent System with Google ADK & A2A Protocol** in **`src/agents/adk_a2a_orchestrator.py`**:

- **`WMSInventoryATPAgent`** (**`wms_inventory_agent`**): Technical WMS/POS agent auditing True ATP and executing optimal split-shipment rerouting.

- **`MerchandisingMarginOTIFAgent`** (**`merchandising_margin_agent`**): Commercial Merchandising & Vendor Compliance agent enforcing the 15% gross margin floor and claiming **`$98,250.00`** in OTIF chargebacks.
- **`RetailPulseOrchestratorAgent`**: Root orchestrator coordinating A2A handoffs between the Technical WMS agent and the Commercial Merchandising agent.
- Standard **A2A Discovery Card** (**`GET /.well-known/agent.json`** & **`GET /api/v1/a2a/agent-card`**) and **A2A JSON-RPC 2.0 Task Endpoint** (**`POST /api/v1/a2a`** & **`POST /api/v1/a2a/tasks/send`** supporting **`tasks/send`** and **`message/send`**).

3. Test the multi-agent hierarchy interactively on your laptop using the **ADK Development UI (`adk web`)** via **`./scripts/run_adk_web.sh`** (**`adk_agents/retail_pulse/agent.py`**).
4. Provision **Google Cloud Run + Vertex AI + Gemini Enterprise** infrastructure using **Terraform** (**`infra/terraform/`**) and deploy/register the A2A agent via **`./labs/lab-04-multi-agent-capstone/deploy_gcp.sh`** and **`./deploy/register_gemini_enterprise_agent.sh`**.

3. Architecture Diagram: ADK + A2A + Cloud Run + Gemini Enterprise

4. Step-by-Step Instructions

Step 1: Initialize the Lab 04 Starter State

Run the Lab 04 installer to ensure Labs 01, 02, and 03 solutions are applied and load the Lab 04 starter stubs (**`src/services/fulfillment_router.py`** and **`src/agents/adk_a2a_orchestrator.py`**):

```
./labs/lab-04-multi-agent-capstone/install.sh
```

Run the verification script to observe the failing Lab 04 capstone tests (**`NotImplementedError`** in **`fulfillment_router.py`** and **`adk_a2a_orchestrator.py`**):

```
./labs/lab-04-multi-agent-capstone/verify.sh
```

Step 2: Implement Smart Split-Shipment Routing (`src/services/fulfillment_router.py`)

Open **`src/services/fulfillment_router.py`** and implement:

1. **`evaluate_candidates(conn, sku, qty, effective_unit_price_usd)`**:

- Query **`wms_inventory`** for all nodes holding **`sku`**.
- Compute **`true_atp`** using **`calculate_node_atp(dict(row))`** from **`src.services.inventory_atp`**.
- Filter to eligible nodes where **`true_atp >= qty`**.
- Compute **`shipping_cost_usd = round(float(row["shipping_zone_cost_usd"]), 2)`**, **`gross_margin_usd = (effective_unit_price_usd - unit_cost_usd) * qty`**, and **`post_shipping_net_margin_usd = round(gross_margin_usd - shipping_cost_usd, 2)`**.
- Sort eligible candidates by **`(-c["post_shipping_net_margin_usd"], c["shipping_cost_usd"], 0 if c["location_type"] == "DC" else 1)`**.

2. **`optimize_at_risk_orders(conn, dry_run=False)`**:

- Query **`omnichannel_orders`** where **`status = 'AT_RISK_STOCKOUT'`** ordered by **`customer_ltv_usd DESC, order_id ASC`**.
- Reroute each order to **`candidates[0]`** (**`status = 'REROUTED_OPTIMAL'`**) and decrement **`on_hand_qty`** in **`wms_inventory`** when **`not dry_run`**.

Step 3: Implement the Google ADK & A2A Multi-Agent Orchestrator (`src/agents/adk_a2a_orchestrator.py`)

Open **`src/agents/adk_a2a_orchestrator.py`** and review how the multi-agent system bridges the **Technical** and **Commercial** personas:

1. **`WMSInventoryATPAgent.execute(conn, dry_run)`**:
- Runs **`audit_vip_oversell_risk(conn)`** and **`optimize_at_risk_orders(conn, dry_run=dry_run)`**.
- Emits an **`A2AAgentHandoff`** from **`wms_inventory_agent`** to **`merchandising_margin_agent`** with the rerouted order count, VIP LTV protected, and preserved post-shipping net margin.
2. **`MerchandisingMarginOTIFAgent.execute(conn, wms_handoff, dry_run)`**:
- Consumes the WMS handoff artifact, runs **`audit_margin_hold_orders(conn)`** (enforcing the 15% gross margin floor), and evaluates all breaching **`vendor_otif_contracts`** (**`actual_otif_pct < contracted_otif_pct`**) to compute the **`$98,250.00`** chargeback recovery.
3. **`RetailPulseOrchestratorAgent.run_orchestration(conn, prompt, dry_run, session_id)`**:
- Coordinates **`WMSInventoryATPAgent`** -> **`MerchandisingMarginOTIFAgent`**, builds the Google ADK agent hierarchy (**`build_adk_agent_hierarchy()`** using **`google.adk.agents.LlmAgent`** and **`SequentialAgent`**), and returns the unified A2A task payload.
4. **A2A Protocol Endpoints (`build_a2a_agent_card` & `handle_a2a_jsonrpc`)**:
- **`GET /.well-known/agent.json`** and **`GET /api/v1/a2a/agent-card`**: Returns the A2A Agent Card declaring **`retail_Margin_Floor_Enforcement`**, **`retail_True_ATP_Omnichannel_Routing`**, and **`retail_Vendor_OTIF_Chargeback_Audit`** skills.
- **`POST /api/v1/a2a`** and **`POST /api/v1/a2a/tasks/send`**: Processes A2A JSON-RPC 2.0 **`tasks/send`** and **`message/send`** requests and returns completed A2A Task objects with **`artifacts`** (**`wms_atp_routing_report`**, **`merchandising_margin_otif_report`**, and **`a2a_handoff_trace`**).

Step 4: Test Locally with the ADK Development UI (`adk web`)

Google ADK includes a built-in interactive web UI (**`adk web`**) for inspecting multi-agent tool calls, sub-agent transfers, and session state locally before deploying to Cloud Run.

1. Verify the ADK Web configuration in dry-run mode:

```
./scripts/run_adk_web.sh --dry-run
```

2. Launch the interactive ADK Development UI on port **`8080`**:

```
./scripts/run_adk_web.sh --port 8080
```

3. Open **`http://127.0.0.1:8080`** in your browser (or via **`/browser`**), select the **`retail_pulse`** agent from the top-left dropdown, and test prompts such as:
- *"Run a dry-run omnichannel triage across WMS True ATP routing, 15% promotional margin floors, and vendor OTIF chargebacks."*
- *"Check the True ATP across all nodes for SKU-1001 and explain the VIP oversell risk."*

- *"Execute live omnichannel remediation: enforce the 15% margin floor, reroute all 35 VIP orders, and claim OTIF vendor chargebacks."*

Step 5: Deploy to Google Cloud Run via Terraform & Register in Gemini Enterprise

The repository includes production-ready **Terraform Infrastructure-as-Code** (**`infra/terraform/`**), a multi-stage Cloud Run **`Dockerfile`**, and deployment scripts (**`deploy/deploy_gcp.sh`** and **`deploy/register_gemini_enterprise_agent.sh`**).

1. **Test the Cloud Run + Terraform + Gemini Enterprise pipeline in `--dry-run` mode** (no GCP credentials required):

```
 ./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run --project-id my-retail-gcp \
-project --region us-central1 --gemini-app-id retail-pulse-enterprise-app
```

2. **Deploy live to Google Cloud Platform** (when authenticated with **`gcloud auth login`**):

```
 ./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --project-id YOUR_GCP_PROJECT_ID --r \
egion us-central1 --gemini-app-id YOUR_GEMINI_ENTERPRISE_APP_ID
```

This script automatically:

- Builds and pushes the container image to **Artifact Registry** via **Cloud Build**
- Runs **`terraform init && terraform apply`** in **`infra/terraform/`** to provision the **Cloud Run v2 service** (**`retail-pulse-a2a-service`**), dedicated least-privilege IAM Service Account, Vertex AI permissions, and environment variables (**`GOOGLE_GENAI_USE_VERTEXAI=TRUE`**)
- Registers the deployed Cloud Run A2A Agent (**`https://<cloud-run-url>/.well-known/agent.json`** & **`/api/v1/a2a`**) with **Gemini Enterprise (Discovery Engine Agent Registry)** via **`deploy/register_gemini_enterprise_agent.sh`**.

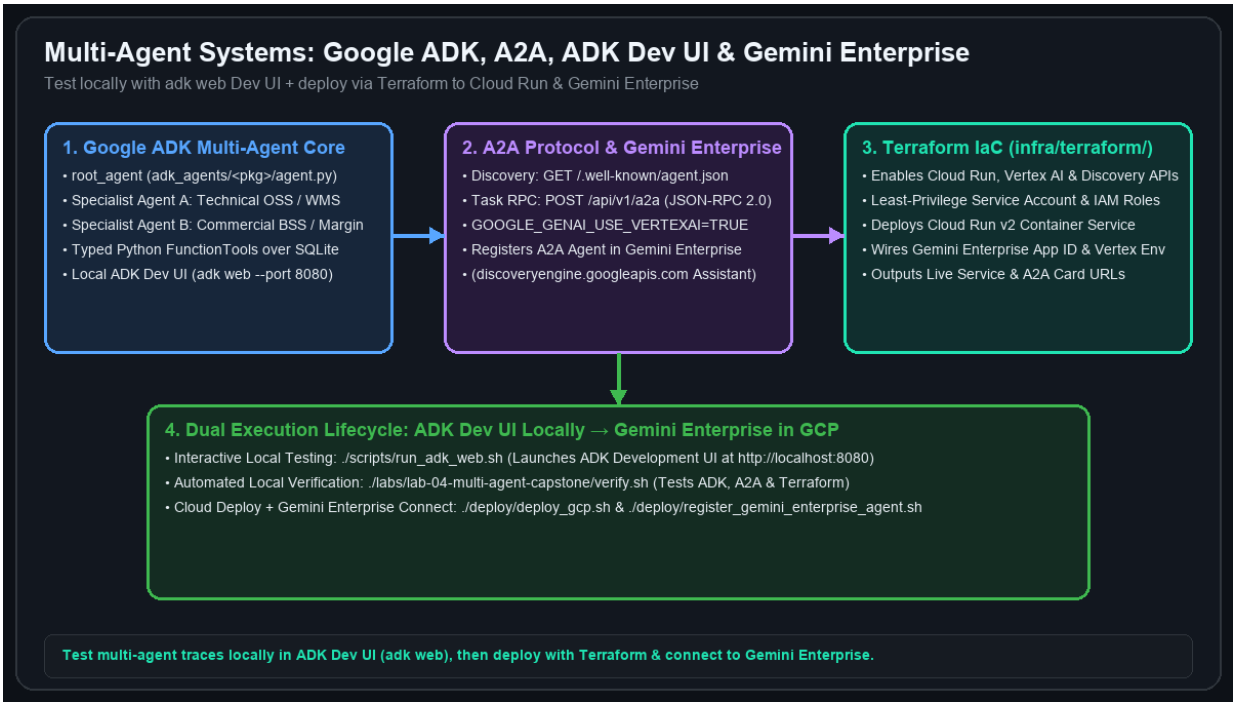
5. Verification Checklist

Run the full capstone verification script (which runs all **20 automated tests** across Labs 01–04):

```
./labs/lab-04-multi-agent-capstone/verify.sh
```

- [ ] **`./labs/lab-04-multi-agent-capstone/verify.sh`** passes all **20/20 automated tests** across **`test_lab01_triage.py`**, **`test_lab02_rules_skills.py`**, **`test_lab03_mcp_hooks.py`**, and **`test_lab04_capstone.py`**.
- [ ] All 35 VIP orders in **`AT_RISK_STOCKOUT`** are transitioned to **`REROUTED_OPTIMAL`** with **`$14,774.00`** total post-shipping net margin.
- [ ] **`GET /.well-known/agent.json`** returns a valid A2A Agent Card and **`POST /api/v1/a2a`** executes the multi-agent ADK handoff (**`wms_inventory_agent -> merchandising_margin_agent`**).
- [ ] **`./scripts/run_adk_web.sh --dry-run`** and **`./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`** succeed cleanly.

Need to apply the verified capstone solution directly? Run **`./labs/lab-04-multi-agent-capstone/apply_solution.sh`**.


Multi-Agent Systems: Google ADK, A2A, ADK Dev UI & Gemini Enterprise
Test locally with adk web Dev UI + deploy via Terraform to Cloud Run & Gemini Enterprise
1. Google ADK Multi-Agent Core
• root_agent (adk_agents/<pkg>/agent.py)
• Specialist Agent A: Technical OSS / WMS
• Specialist Agent B: Commercial BSS / Margin
• Typed Python FunctionTools over SQLite
• Local ADK Dev UI (adk web --port 8080)
2. A2A Protocol & Gemini Enterprise
• Discovery: GET /.well-known/agent.json
• Task RPC: POST /api/v1/a2a (JSON-RPC 2.0)
• GOOGLE_GENAI_USE_VERTEXAI=TRUE
• Registers A2A Agent in Gemini Enterprise
• (discoveryengine.googleapis.com Assistant)
3. Terraform IaC (infra/terraform/)
• Enables Cloud Run, Vertex AI & Discovery APIs
• Least-Privilege Service Account & IAM Roles
• Deploys Cloud Run v2 Container Service
• Wires Gemini Enterprise App ID & Vertex Env
• Outputs Live Service & A2A Card URLs
4. Dual Execution Lifecycle: ADK Dev UI Locally → Gemini Enterprise in GCP
• Interactive Local Testing: ./scripts/run_adk_web.sh (Launches ADK Development UI at http://localhost:8080)
• Automated Local Verification: ./labs/lab-04-multi-agent-capstone/verify.sh (Tests ADK, A2A & Terraform)
• Cloud Deploy + Gemini Enterprise Connect: ./deploy/deploy_gcp.sh & ./deploy/register_gemini_enterprise_agent.sh
Test multi-agent traces locally in ADK Dev UI (adk web), then deploy with Terraform & connect to Gemini Enterprise.

LAB 04 COPY-PASTE ANTIGRAVITY PROMPTS (EN)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Lab 04 Prompt Playbook: Google ADK, A2A Protocol, Omnichannel Routing & Gemini Enterprise Deployment

Copy and paste these prompts into **Google Antigravity** as you complete the Lab 04 Capstone. Each phase includes prompts for both the **Technical Track** and the **Commercial / Executive Track**.

Phase 1: Discovery & Multi-Agent Planning (`/plan` & `/boost`)

Prompt 1A — Technical Track: Deep Architectural Plan for Routing + ADK + A2A (`/plan`)

```
/plan Read @src/services/fulfillment_router.py, @src/agents/adk_a2a_orchestrator.py, @ad \
  k_agents/retail_pulse/agent.py, @src/main.py, and @tests/test_lab04_capstone.py.
Create a step-by-step implementation plan to complete our Lab 04 Capstone:
1. In `src/services/fulfillment_router.py`, implement `evaluate_candidates` and `optimiz \
  e_at_risk_orders` to rank nodes with `true_atp >= qty` by `(-post_shipping_net_margin_ \
  usd, shipping_cost_usd, 0 if location_type == 'DC' else 1)` and reroute all 35 `AT_RIS \
  K_STOCKOUT` VIP orders to `REROUTED_OPTIMAL`.
2. In `src/agents/adk_a2a_orchestrator.py`, implement:
   - `WMSInventoryATPAgent.execute(conn, dry_run)` (True ATP audit + optimal split-shipm \
  ent routing + `A2AAgentHandoff` to `merchandising_margin_agent`)
   - `MerchandisingMarginOTIFAgent.execute(conn, wms_handoff, dry_run)` (15% gross margi \
  n floor enforcement + vendor OTIF chargeback recovery)
   - `RetailPulseOrchestratorAgent.run_orchestration(conn, prompt, dry_run, session_id)` \
   and `build_adk_agent_hierarchy()` using `google.adk.agents.LlmAgent` and `SequentialA \
  gent`
   - `build_a2a_agent_card(base_url)` and `handle_a2a_jsonrpc(conn, rpc_request, base_ur \
  l)` for `GET /.well-known/agent.json` and `POST /api/v1/a2a` (`tasks/send` and `messag \
  e/send`).
```

Prompt 1B — Commercial / Operations Track: Trade-Off & Multi-Agent Governance Analysis (`/boost`)

```
/boost Read @src/services/fulfillment_router.py, @src/agents/adk_a2a_orchestrator.py, an \
  d @tests/test_lab04_capstone.py.
From a retail omnichannel profitability perspective, analyze why separating responsibili \
  ties between a Technical `WMSInventoryATPAgent` (True ATP & DC-first post-shipping mar \
  gin routing) and a Commercial `MerchandisingMarginOTIFAgent` (15% gross margin floor & \
   vendor OTIF debit memos) connected via the A2A Protocol prevents both negative-margin \
   order bleed and VIP stockout cancellations. Then implement both modules so all 20 wor \
  kshop tests pass.
```

Phase 2: Multi-Agent Execution (`/teamwork` & Subagents)

Prompt 2A — Parallel Subagent Orchestration (`/teamwork`)

```
/teamwork Coordinate three specialized workstreams to complete our Lab 04 Capstone:
1. Omnichannel Routing Engineer: Implement `evaluate_candidates` and `optimize_at_risk_o \
  rders` in @src/services/fulfillment_router.py so all 35 `AT_RISK_STOCKOUT` VIP orders  \
  are rerouted to the highest post-shipping net margin node with sufficient True ATP.
2. ADK & A2A Systems Architect: Implement `WMSInventoryATPAgent`, `MerchandisingMarginOT \
  IFAgent`, `RetailPulseOrchestratorAgent`, `build_a2a_agent_card`, and `handle_a2a_json \
  rpc` in @src/agents/adk_a2a_orchestrator.py.
3. QA, ADK Web & Cloud Run Auditor: Run `./labs/lab-04-multi-agent-capstone/verify.sh` t \
  o validate all 20 tests across Labs 01-04, verify `./scripts/run_adk_web.sh --dry-run` \
   and `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`, and produce an exec \
  utive summary table of the financial recovery.
```

Phase 3: Inline Edit (`Cmd+I` / `Ctrl+I`)

Open **`src/services/fulfillment_router.py`**, select **`evaluate_candidates`** and **`optimize_at_risk_orders`**, press **`Cmd+I`** (macOS) or **`Ctrl+I`** (Windows/Linux), and enter:

```
Implement `evaluate_candidates` using `calculate_node_atp` to filter nodes with `true_at \
  p >= qty`, compute `post_shipping_net_margin_usd = round((effective_unit_price_usd - u \
  nit_cost_usd) * qty - shipping_cost_usd, 2)`, and sort by `(-post_shipping_net_margin_ \
  usd, shipping_cost_usd, 0 if location_type == 'DC' else 1)`. Then implement `optimize_ \
  at_risk_orders` to reroute `AT_RISK_STOCKOUT` orders (ordered by `customer_ltv_usd DES \
  C, order_id ASC`) to `candidates[0]`, updating `omnichannel_orders` (`status='REROUTED \
  _OPTIMAL'`) and decrementing `wms_inventory.on_hand_qty` when `not dry_run`.
```

Phase 4: Local Interactive Testing with adk web & `/browser`

Prompt 4A — Test Multi-Agent Hierarchy Locally in ADK Web UI (adk web)

```
Validate our local Google ADK Development UI setup by running `./scripts/run_adk_web.sh  \
  --dry-run`, and inspect @adk_agents/retail_pulse/agent.py to confirm how `root_agent`  \
  (`retail_pulse_root_agent`) delegates across `wms_inventory_agent`, `merchandising_mar \
  gin_agent`, and `a2a_bridge_agent`.
```

Prompt 4B — End-to-End Browser Verification of the Retail Pulse Command Center (`/browser`)

```
/browser Start the FastAPI server (`uvicorn src.main:app --port 8000`) if it is not alre \
  ady running, open `http://127.0.0.1:8000`, and verify:
1. The header displays the Tryolabs logo ("Delivered by Tryolabs") and the Retail Pulse  \
  Command Center KPIs.
2. Click "1. Enforce 15% Margin Floor (Lab 01)", "2. Optimize Split-Shipment Routing (La \
  b 04)", and "3. Claim OTIF Chargebacks (Lab 02/04)".
3. Confirm that Margin Hold Orders = 0, VIP Stockout Risk = 0, and all 6 breaching vendo \
  r contracts show `CHARGEBACK_CLAIMED` totaling $98,250.00.
4. Also fetch `http://127.0.0.1:8000/.well-known/agent.json` to inspect the live A2A Age \
  nt Card.
```

Phase 5: Cloud Run + Terraform Deployment & Gemini Enterprise Registration

Prompt 5 — Validate Terraform Infrastructure & Gemini Enterprise A2A Registration

```
Read @infra/terraform/main.tf, @deploy/deploy_gcp.sh, and @deploy/register_gemini_enterp \
  rise_agent.sh.
Run `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run --project-id retail-puls \
  e-prod --region us-central1 --gemini-app-id retail-pulse-ge-app` and explain each step \
   of the deployment pipeline:
1. Building the container image with Cloud Build & Artifact Registry
2. Provisioning Cloud Run v2, least-privilege IAM Service Account, and Vertex AI binding \
  s with Terraform
3. Registering the Cloud Run A2A Agent Card (`/.well-known/agent.json` & `/api/v1/a2a`)  \
  in Gemini Enterprise (Discovery Engine).
```